

PostgreSQL 19 新機能概要

Noriyoshi Shinoda

June 27, 2026

Speaker

- ✓篠田 典良(しのだ のりよし)
- ✓所属
 - ✓日本ヒューレット・パカード合同会社
- ✓現在の業務など
 - ✓PostgreSQL 開発 (PostgreSQL 10~18, 19 dev)
 - ✓「篠田の虎の巻」作成
 - ✓PostgreSQLをはじめ Oracle Database, Microsoft SQL Server, Vertica 等 RDBMS 全般に関するシステムの設計、移行、チューニング、コンサルティング
 - ✓Oracle ACE Pro (2009~)
- ✓関連する URL
 - ✓「PostgreSQL 虎の巻」シリーズ
<https://github.com/nori-shinoda/documents/blob/main/README.md>
 - ✓Redgate 100 in 2022 (Most influential in the database community 2022)
<https://www.red-gate.com/hub/redgate-100/>
 - ✓Oracle ACE Profile
<https://ace.oracle.com/apex/ace/profile/nshino483>



Featured in
The Redgate 100



日本 PostgreSQL ユーザ会 (JPUG)

JPUG とは？

- ✓ PostgreSQL の普及を促進するための特定非営利活動法人 (NPO 法人)
- ✓ 技術情報の公開やユーザ相互の情報交換を支援
- ✓ メーリングリストや Slack の運用
- ✓ 勉強会、カンファレンスの開催や開発支援
- ✓ マニュアルの翻訳



PostgreSQL とは？

PostgreSQL 概要

- ✓ PostgreSQL とは？
 - ✓ オープンソースで開発される RDBMS
 - ✓ 所有する企業を持たない
 - ✓ ライセンスは PostgreSQL ライセンス (BSD ライセンス, MIT ライセンスに近い)
- ✓ PostgreSQL をベースに多くの派生 RDBMS が誕生
 - ✓ Google Cloud AlloyDB for PostgreSQL
 - ✓ Amazon Aurora PostgreSQL
 - ✓ Amazon Redshift
 - ✓ EDB Postgres Advanced Server
 - ✓ Yugabyte YugabyteDB
 - ✓ Lakebase Postgres
 - ✓ Tsurugi
 - ✓ Apache Cloudberry (Greenplum)



PostgreSQL の開発スケジュール

メジャー・バージョンアップ

- ✓ 最新バージョンは PostgreSQL 18 (18.4)
 - ✓ 18 = メジャー・バージョン、4 = マイナー・バージョン
- ✓ 年に1回メジャー・バージョンアップ
 - ✓ 4 月 = Feature Freeze
 - ✓ 5 月～9月 = Beta 期間
 - ✓ 9 月 = RC 期間
 - ✓ 9 月末 = 正式リリース
- ✓ マイナー・バージョンアップ
 - ✓ バグフィックスと、脆弱性対応がメイン
 - ✓ 3 か月に1回(例外あり)
 - ✓ 旧 4 メジャー・バージョンに対して適用
- ✓ 開発中のバージョンは PostgreSQL 19 Beta 1 (19.0)
 - ✓ 本日の話題
 - ✓ 正式バージョンまでに変更される可能性があります

主要な新機能



パーティションのマージと分割

パーティション・テーブルとは？

- ✓パーティション・テーブルは巨大なテーブルを物理分割する方法
 - ✓親となるパーティション・テーブルと、実データが格納される複数のパーティションで構成
- ✓アプリケーションはパーティションの存在を意識しない
 - ✓WHERE 句から特定のパーティションに自動転送(パーティション・プルーニング)
- ✓テーブル分割方法は、RANGE, LIST, HASH の3種類
- ✓パーティション・テーブル作成例

```
postgres=> CREATE TABLE part1(col1 INT, col2 VARCHAR(10)) PARTITION BY RANGE(col1);  
CREATE TABLE  
postgres=> CREATE TABLE part1v1 PARTITION OF part1 FOR VALUES FROM (0) TO (1000000);  
CREATE TABLE  
postgres=> CREATE TABLE part1v2 PARTITION OF part1 FOR VALUES FROM (1000000) TO  
(2000000);  
CREATE TABLE
```

パーティションのマージと分割

パーティションの分割

- ✓ PostgreSQL 19 ではパーティションの分割をサポート
 - ✓ 内部的には「**新しいテーブルの作成 + データコピー + テーブル名の変更**」が発生する
 - ✓ 元のパーティションに作成されたインデックスは削除される
 - ✓ オプティマイザ統計は再取得される
- ✓ パーティションの分割例

```
postgres=> ALTER TABLE part3 SPLIT PARTITION part1v1 INTO (  
            PARTITION part3v2 FOR VALUES FROM (0) TO (500000),  
            PARTITION part3v3 FOR VALUES FROM (500000) TO (1000000));  
  
ALTER TABLE
```


パーティションのマージと分割

パーティションのマージ(統合)

- ✓ PostgreSQL 19 ではパーティションのマージをサポート
 - ✓ 内部的には「**新しいテーブルの作成+データコピー+テーブル名の変更**」が発生する
 - ✓ 元のパーティションに作成されたインデックスは削除される
 - ✓ オプティマイザ統計は再取得される
- ✓ パーティションのマージ例

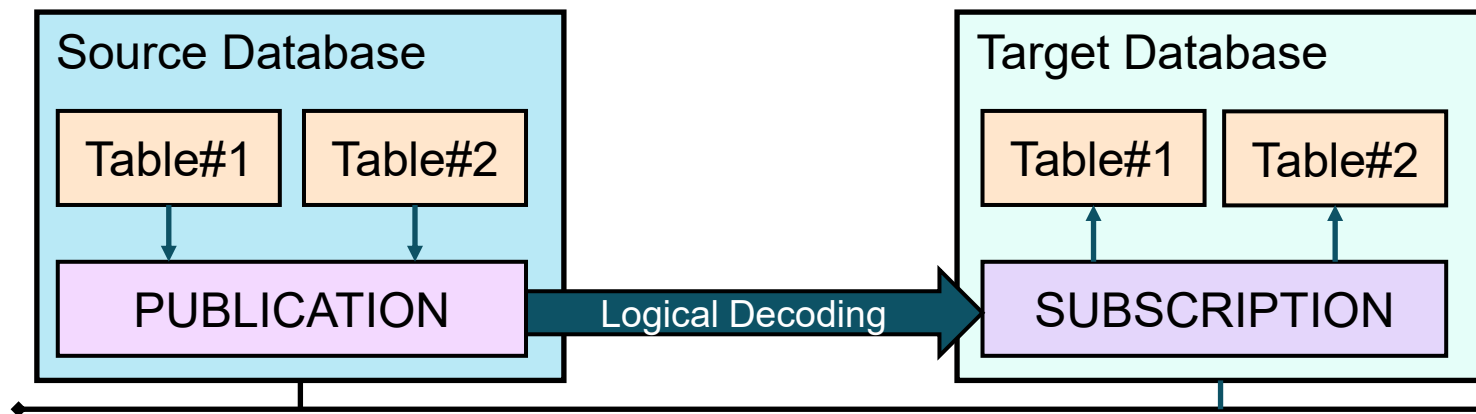
```
postgres=> ALTER TABLE part3 MERGE PARTITIONS (part2v1, part2v2) INTO part2v3;  
ALTER TABLE
```

- ✓ 構文上の注意
 - ✓ パーティションの分割は SPLIT PARTITION、パーティションのマージは MERGE PARTITION**S** になる

ロジカルレプリケーション

ロジカルレプリケーションとは？

- ✓ テーブル単位のレプリケーション機能
 - ✓ レプリケーション先テーブルも更新可能
 - ✓ 異なるバージョン間や、テーブル定義が若干異なってもレプリケーション可能
- ✓ パブリケーション(PUBLICATION)とサブスクライバ(SUBSCRIPTION)を作成
 - ✓ PUBLICATION = データ供給側(データ送信対象テーブルの決定)
 - ✓ WAL sender プロセスが論理デコードされた差分情報をサブスクライバに転送
 - ✓ SUBSCRIPTION = データ受信側(接続先サーバ、PUBLICATION の決定)
 - ✓ WAL receiver プロセスがデコードされた差分をテーブルに適用



ロジカルレプリケーション

シーケンスのレプリケーション

- ✓シーケンスのレプリケーション機能

- ✓シーケンス値を同期する(従来はテーブルだけ)

- ✓制約

- ✓専用の PUBLICATION を作成する必要がある、個別のシーケンスを指定することはできない

- ✓シーケンスの同期は手動(ALTER SUBSCRIPTION REFRESH SEQUENCES 文を実行)

- ✓シーケンスの同期例

```
postgres=# CREATE PUBLICATION pub1 FOR ALL SEQUENCES;  
ALTER TABLE
```

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION 'host=...' PUBLICATION pub1;  
CREATE SUBSCRIPTION  
postgres=# ALTER SUBSCRIPTION sub1 REFRESH SEQUENCES;  
ALTER SUBSCRIPTION
```

ロジカルレプリケーション

接続先の指定構文

- ✓サブスクライバの接続先設定
- ✓従来の構文

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION 'host=dbsvr1 port=5432 dbname=postgres
          user=postgres ...' PUBLICATION pub1;
CREATE SUBSCRIPTION
```

- ✓新しい構文
 - ✓接続先の情報を SERVER オブジェクトで管理できるように

```
postgres=# CREATE EXTENSION postgres_fdw;
postgres=# CREATE SERVER svr1 FOREIGN DATA WRAPPER postgres_fdw OPTIONS (...);
postgres=# CREATE USER MAPPING FOR ...;
postgres=# CREATE SUBSCRIPTION sub1 SERVER svr1 PUBLICATION pub1;
```

ロジカルレプリケーション

サブスクライバの属性

✓サブスクライバの属性

✓retain_dead_tuples オプション(デフォルト False)

✓true に設定するとリモート・トランザクションが完了するまで削除済のレコードを維持する

✓update_deleted 競合の検知が有効になる

✓パラメーター track_commit_timestamp の有効化もあわせて必要

✓max_retention_duration オプション(デフォルト 0)

✓競合検出用に保持される時間の最大値

✓設定例

```
postgres=# CREATE SUBSCRIPTION sub1 CONNECTION 'port=5432 dbname=postgres' PUBLICATION pub1
          WITH (retain_dead_tuples = true, max_retention_duration = 10) ;
NOTICE:  created replication slot "sub1" on publisher
CREATE SUBSCRIPTION
```

SQL Property Graph

グラフの作成

- ✓ISO/IEC 9075-16:2023 で規格作成
- ✓CREATE PROPERTY GRAPH 文で作成

```
postgres=> CREATE TABLE member (member_id INT PRIMARY KEY, member_name TEXT) ;
CREATE TABLE
postgres=> CREATE TABLE follow (member_id1 INT, member_id2 INTEGER,
                                PRIMARY KEY(member_id1, member_id2)) ;
CREATE TABLE
postgres=> CREATE PROPERTY GRAPH graph1
    VERTEX TABLES (member KEY (member_id) PROPERTIES (member_id, member_name))
    EDGE TABLES (follow
        SOURCE KEY (member_id1) REFERENCES member(member_id)
        DESTINATION KEY (member_id2) REFERENCES member(member_id)) ;
CREATE PROPERTY GRAPH
```

SQL Property Graph

グラフの検索

✓GRAPH_TABLE 関数で検索を行う

```
postgres=> SELECT * FROM GRAPH_TABLE (graph1 MATCH
      (mem1 IS member)-[follow1 IS follow]->(mem2 IS member)
      -[follow2 IS follow]->(mem3 IS member)
      WHERE mem1.member_id = mem3.member_id COLUMNS
      (mem1.member_id, mem2.member_id)) ;
```

member_id		member_id
100		200
200		100

(2 rows)

ARCHITECTURE



I/O Worker

非同期 I/O ワーカー数の自動調整機能

- ✓非同期 I/O Worker 数の自動調整
 - ✓PostgreSQL 18 で追加された I/O Worker
 - ✓PostgreSQL 19 ではワーカー数が自動調整される
- ✓パラメーター設定

PostgreSQL 18	PostgreSQL 19	説明
io_method (worker)	io_method (worker)	I/O ワーカーの実装
io_workers (3)	io_min_workers (2)	I/O ワーカーの最小数
	io_max_workers (8)	I/O ワーカーの最大数
	io_worker_idle_timeout (1min)	I/O ワーカーを縮小するタイムアウト時間
	io_worker_launch_interval (100ms)	I/O ワーカーを起動する最小間隔



自動 Vacuum

実行順位決定機能

- ✓ 閾値からの乖離率(スコア)を計算し、乖離が大きいテーブルから VACUUM を実施
 - ✓ 従来は pg_class カタログを順番に走査
 - ✓ pg_stat_autovacuum_scores ビューで確認
 - ✓ 例えば VACUUM の閾値が 80 で、実際に更新されたレコード数が 100 の場合、スコアは 1.25 になる
- ✓ 実際のスコアは以下のパラメーターの乗算になる

パラメーター名(デフォルト値)	説明
autovacuum_analyze_score_weight (1.0)	ANALYZE 処理のウェイト
autovacuum_freeze_score_weight (1.0)	FREEZE 処理のウェイト
autovacuum_multixact_freeze_score_weight (1.0)	マルチランザクションFREEZE 処理のウェイト
autovacuum_vacuum_insert_score_weight (1.0)	INSERT による VACUUM 処理のウェイト
autovacuum_vacuum_score_weight (1.0)	VACUUM 処理のウェイト



自動 Vacuum

並列処理

- ✓ 自動パラレル VACUUM
 - ✓ VACUUM 文の PARALLEL オプションと同等の処理
 - ✓ パラメーター autovacuum_max_parallel_workers (デフォルト値 0) で並列度を決定
 - ✓ テーブル単位には autovacuum_parallel_workers 設定を変更
- ✓ パラレル VACUUM 自体は PostgreSQL 13 から利用可能

```
postgres=> VACUUM (PARALLEL 2, VERBOSE) data1;  
INFO:  vacuuming "postgres.public.data1"  
INFO:  launched 1 parallel vacuum worker for index vacuuming (planned: 1)  
INFO:  finished vacuuming "postgres.public.data1": index scans: 1  
. . .
```

SQL Statement



REPACK

REPACK 文とは？

- ✓データの物理的な並べ替えと、不要データの削除を行う
 - ✓従来は VACUUM FULL 文または CLUSTER 文で実行していた
 - ✓従来は長時間 **ACCESS EXCLUSIVE ロック**が必要だった
- ✓MAINTAIN 権限を持つテーブルに対して実行可能
- ✓実行例

```
postgres=> REPACK (VERBOSE) data1;  
INFO:  repacking "public.data1" in physical order  
INFO:  "public.data1": found 0 removable, 30000000 nonremovable row versions in 162163  
pages  
DETAIL:  0 dead row versions cannot be removed yet.  
CPU: user: 0.18 s, system: 4.24 s, elapsed: 54.25 s.  
REPACK
```

REPACK

REPACK 文のオプション

✓オプション

オプション名	説明
ANALYZE	オプティマイザ統計の再取得を行う
VERBOSE	詳細なログを出力する
CONCURRENTLY	ロックを最小限にして他のSQL文のアクセスを許可
USING INDEX	指定されたインデックスを使用して再構築

✓CONCURRENTLY オプション

- ✓実行中は差分を論理デコードして一時保管する(pgrepack プラグインを使用)
- ✓WAL 出力レベルが一時的に上昇する(**effective_wal_level** が replica から logical に変更)

✓ステータス確認

- ✓REPACK 実行状況は pg_stat_progress_repack ビューで確認できる

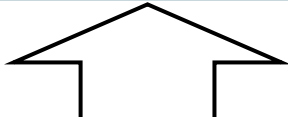


FOR PORTION OF

範囲型の部分更新構文

- ✓DELETE 文または UPDATE 文で使用する
 - ✓範囲型の特定の範囲だけ UPDATE / DELETE 文を実行する
 - ✓範囲型 (daterange, int4range, numrange 等) 列を指定する
 - ✓実行により、レコードが増える可能性あり

```
postgres=> UPDATE range1 FOR PORTION OF col1 FROM '2026-04-01' TO '2026-04-30'  
           SET col2 = 200;  
UPDATE 1
```



col1 (daterange)	col2 (int)
2026/03/01-2026/05/30	100



col1 (daterange)	col2 (int)
2026/03/01-2026/03/30	100
2026/04/01-2026/04/30	200
2026/05/01-2026/05/30	100

COPY FROM

オプションの追加

- ✓ヘッダーのレコード数を指定できる(**HEADER** オプション／従来は行数指定無し)
- ✓エラー発生時に対象列値を NULL に指定できる(**ON_ERROR** オプション)

```
postgres=# COPY data1 FROM stdin WITH (HEADER 2, ON_ERROR set_null, FORMAT csv) ;
```

```
Enter data to be copied followed by a newline.
```

```
End with a backslash and a period on a line by itself, or an EOF signal.
```

```
>> HEADER1                ← ヘッダ 1行目
```

```
>> HEADER2                ← ヘッダ 2行目
```

```
>> 100, error_data        ← col2 列はエラーになるので NULL になる
```

```
>> 200, 200
```

```
>> ¥.
```

```
NOTICE:  in 1 row, columns were set to null due to data type incompatibility
```

```
COPY 2
```


COPY TO

オプションの追加

- ✓パーティション・テーブル指定可能
- ✓JSON 形式の出力 (FORMAT オプション)

```
postgres=# COPY data1 TO stdout WITH (FORMAT JSON, FORCE_ARRAY);  
[  
  {"col1":100,"col2":null}  
, {"col1":200,"col2":200}  
]
```

INSERT

ON CONFLICT DO SELECT

✓ON CONFLICT 句に DO SELECT 句を指定可能

✓実行例

```
postgres=> INSERT INTO data1 VALUES (100, 'conf11') ON CONFLICT (col1)
           DO SELECT FOR UPDATE RETURNING *;
```

col1	col2
100	data1

(1 row)

```
INSERT 0 1
```

CHECKPOINT

オプションの追加

✓モードの指定

- ✓MODE FAST = チェックポイント処理を即座に実行(デフォルト)

- ✓MODE SPREAD = チェックポイントの負荷を分散し、時間をかけて実行

✓UNLOGGED テーブル情報のフラッシュ

- ✓デフォルトでは UNLOGGED テーブルの情報はストレージにフラッシュされない

- ✓FLUSH_UNLOGGED TRUE を指定することでフラッシュされる

✓実行例

```
postgres=# CHECKPOINT (MODE SPREAD);  
CHECKPOINT
```

```
postgres=# CHECKPOINT (FLUSH_UNLOGGED TRUE);  
CHECKPOINT
```

OTHER

その他の新構文

- ✓GROUP BY ALL
 - ✓全出力列を使って GROUP BY 句を実行
- ✓EXPLAIN (IO)
 - ✓ANALYZE 句と同時に指定して、I/O 統計を出力
- ✓WAIT FOR LSN
 - ✓指定した LSN が実行されるまで待機する



Commands



psql

プロンプトと出力フォーマットの指定

- ✓ PROMPT[123] 変数の設定値
 - ✓ %S = パラメーター search_path の値を出力
 - ✓ %i = パラメーター in_hot_standby の値を出力
- ✓ \pset display_true, display_false 設定
 - ✓ TRUE 値と FALSE 値の出力文字列を指定

```
postgres=> \pset display_true 'True'
Boolean true display is "True".
postgres=> \pset display_false 'False'
Boolean false display is "False".
postgres=> SELECT true::boolean, false::boolean ;
 bool | bool
-----+-----
  True | False
(1 row)
```

pg_dump

オプティマイザ統計のダンプとリストア

- ✓ 拡張統計 (EXTENDED STATS) 情報が出力される
 - ✓ pg_restore_extended_stats 関数が追加
 - ✓ 従来は CREATE STATISTICS 文のみが出力
- ✓ 実行例

```
$ pg_dump --statistics-only | grep restore
SELECT * FROM pg_catalog.pg_restore_relation_stats(
SELECT * FROM pg_catalog.pg_restore_attribute_stats(
SELECT * FROM pg_catalog.pg_restore_attribute_stats(
SELECT * FROM pg_catalog.pg_restore_extended_stats(
SELECT * FROM pg_catalog.pg_restore_relation_stats(
```

Contrib modules



pg_plan_advice

実行計画の変更

- ✓ 実行計画を変更する初めてのモジュール
 - ✓ 従来は pg_hint_plan 等の外部モジュールが必要だった
- ✓ 利用方法
 - ✓ モジュールをロード (LOAD 文、shared_preload_libraries パラメーター)
 - ✓ pg_plan_advice.advice パラメーターに実行計画の変更を指示
 - ✓ アドバイスの指定

指定	備考
SEQ_SCAN(target)	全件検索 (Seq Scan) の実行
INDEX_SCAN(target)	インデックス検索 (Index Scan) の実行
JOIN_ORDER(item1 item2)	結合順序の指定
HASH_JOIN(target)	ハッシュ結合の指定
NESTED_LOOP_PLAN(target)	ネステッド・ループの指定
GATHER(item)	パラレル・クエリーの指定



pg_plan_advice

実行計画の変更

✓例

```
postgres=> EXPLAIN SELECT * FROM d1 INNER JOIN d2 ON d1.c1 = d2.c1;
```

```
...
```

```
-> Seq Scan on d1 (cost=0.00..16.00 rows=1000 width=10)
```

```
-> Hash (cost=16.00..16.00 rows=1000 width=10)
```

```
-> Seq Scan on d2 (cost=0.00..16.00 rows=1000 width=10)
```

```
postgres=> SET pg_plan_advice.advice = 'JOIN_ORDER(d2 d1)';
```

```
SET
```

```
postgres=> EXPLAIN SELECT * FROM d1 INNER JOIN d2 ON d1.c1 = d2.c1;
```

```
...
```

```
-> Seq Scan on d2 (cost=0.00..16.00 rows=1000 width=10)
```

```
-> Hash (cost=16.00..16.00 rows=1000 width=10)
```

```
-> Seq Scan on d1 (cost=0.00..16.00 rows=1000 width=10)
```

Supplied Plan Advice:

```
JOIN_ORDER(d2 d1) /* matched */
```

```
...
```

pg_stash_advice

実行計画の変更グループ

- ✓ pg_plan_advice モジュールの設定をグループ(STASH)化
- ✓ 実行計画のグループを作成し、Query ID 単位に実行計画を指定する

```
postgres=# SELECT pg_create_advice_stash('stash1') ;
pg_create_advice_stash
```

```
postgres=# SELECT pg_set_stashed_advice('stash1', 1524396781679940627,
      'JOIN_ORDER(d2 d1)') ;
pg_set_stashed_advice
```

```
postgres=# SET pg_stash_advice.stash_name = 'stash1' ;
SET
```

非互換



非互換

旧バージョンとの差分

✓PostgreSQL 17 との違い

- ✓MD5 認証は非推奨に(md5_password_warnings パラメーター)
- ✓ブロック・チェックサム機能がデフォルトで有効化
- ✓COPY FROM 文で、stdin 以外の入力データで「¥.」はデータ完了とみなさない
- ✓外部テーブルに対する COPY FREEZE 文の禁止

✓PostgreSQL 18 との違い

- ✓パラメーター standard_conforming_strings 変更不可
- ✓MD5 認証時に警告メッセージ出力(md5_password_warnings = on の場合)
- ✓COPY FROM 文で WHERE 句にシステム列を含むとエラーに
- ✓JSON_ARRAY 関数の戻り値が空の場合、空の配列が返る(従来は NULL)

PostgreSQL 19 には入らなかった機能



PostgreSQL 19 には入らなかった機能

PostgreSQL 20 に期待

- ✓COMMIT されず
 - ✓差分更新マテリアライズド・ビュー
 - ✓リアルタイム更新マテリアライズド・ビュー
 - ✓行パターン認識
 - ✓LET コマンド
- ✓COMMIT 後に Revert
 - ✓ALTER DOMAIN VALIDATE CONSTRAINT 文のロックレベル変更
 - ✓pg_dumpall コマンドのテキスト以外の出力フォーマット

情報



参考になる情報

PostgreSQL 19 情報

- ✓ PostgreSQL 19 新機能検証結果 (篠田の虎の巻)
https://github.com/nori-shinoda/documents/blob/main/postgresql19_beta1_new_features_ja_20260606-1.pdf
- ✓ PostgreSQL 19 への展望 (SRA OSS 高塚さん)
https://www.postgresql.jp/sites/default/files/2026-03/osc_jpug_pg19dev.pdf
- ✓ pgPedia PostgreSQL 19 info
<https://pgpedia.info/postgresql-versions/postgresql-19.html>
- ✓ PostgreSQL 19 Release Notes
<https://www.postgresql.org/docs/19/release-19.html>
- ✓ PostgreSQL 19 Manual
<https://www.postgresql.org/docs/19/index.html>
- ✓ PostgreSQL pgsql-hackers mailing-list
<https://www.postgresql.org/list/pgsql-hackers/>



Thank you

Mail : noriyoshi.shinoda@hpe.com
X(Twitter) : @nori_shinoda
Qiita : @plusultra

